

Network Simulator Tutorial

DC & Computer Networks Lab

Network Simulation *

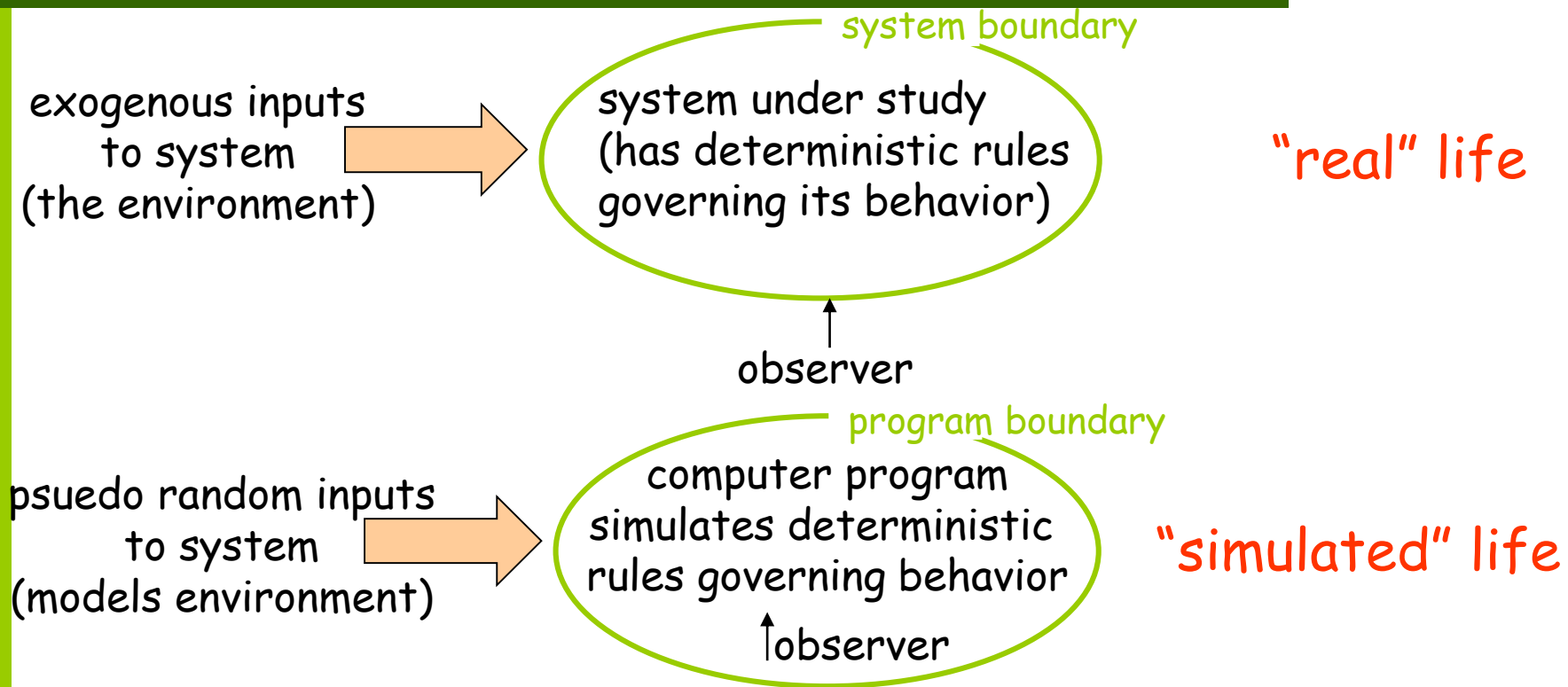
Motivation:

- Learn fundamentals of evaluating network performance via simulation

Overview:

- fundamentals of discrete event simulation
- ns-2 simulation

What is simulation? *



Why Simulation? *

- real-system not *available, is complex/costly or dangerous* (eg: space simulations, flight simulations)
- quickly evaluate design *alternatives* (eg: different system configurations)
- evaluate *complex functions* for which closed form formulas or numerical techniques not available

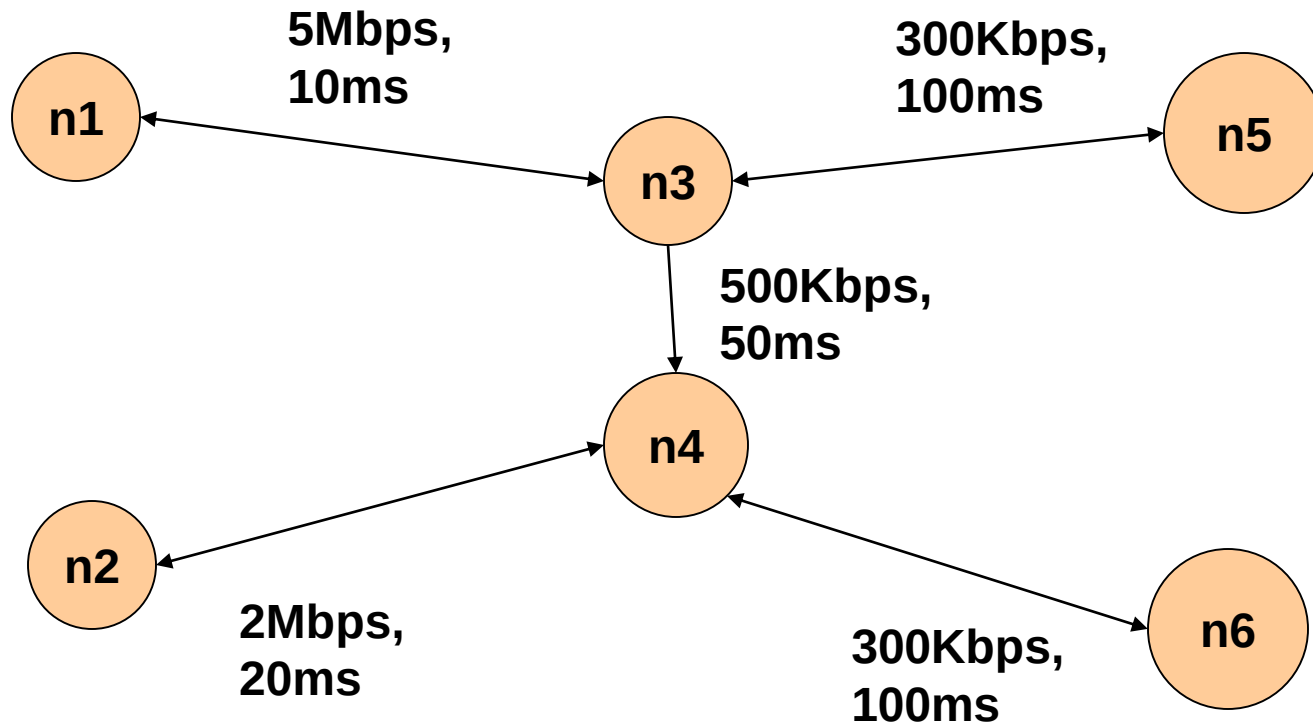
Simulation: advantages/drawbacks*

- advantages:
 - sometimes cheaper
 - find bugs (in design) in advance
 - **generality**: over analytic/numerical techniques
 - **detail**: can simulate system details at arbitrary level
- drawbacks:
 - caution: does model reflect reality
 - large scale systems: lots of resources to simulate (especially accurately simulate)
 - may be slow (computationally expensive – 1 min real time could be hours of simulated time)
 - art: determining right level of model complexity
 - statistical uncertainty in results

What is NS2?

- Network Simulator
- A package of tools that simulates behavior of networks
 - Create Network Topologies
 - Log events that happen under any load
 - Analyze events to understand the network behavior

Creating Topologies



Creating Topologies

- Nodes
 - Set properties like queue length, location
 - Protocols, routing algorithms
- Links
 - Set types of link – Simplex, duplex, wireless, satellite
 - Set bandwidth, latency etc.
- Done through tcl Scripts

Observing Network Behavior

- Observe behavior by tracing “events”
 - Eg. packet received, packet drop etc

Src Dst IP
Address, Port

```
+ 0.1 1 2 cbr 1000 ----- 2 1.0 5.0 0 0
- 0.1 1 2 cbr 1000 ----- 2 1.0 5.0 0 0
r 0.114 1 2 cbr 1000 ----- 2 1.0 5.0 0 0
+ 0.114 2 3 cbr 1000 ----- 2 1.0 5.0 0 0
- 0.114 2 3 cbr 1000 ----- 2 1.0 5.0 0 0
r 0.240667 2 3 cbr 1000 ----- 2 1.0 5.0 0 0
```

time

Trace File

event	time	from node	to node	pkt type	pkt size	flags	fid	src addr	dst addr	seq num	pkt id
-------	------	--------------	------------	-------------	-------------	-------	-----	-------------	-------------	------------	-----------

r : receive (at to_node)

+ : enqueue (at queue)

- : dequeue (at queue)

d : drop (at queue)

src_addr : node.port (3.0)

dst_addr : node.port (0.0)

```
r 1.3556 3 2 ack 40 ----- 1 3.0 0.0 15 201
+ 1.3556 2 0 ack 40 ----- 1 3.0 0.0 15 201
- 1.3556 2 0 ack 40 ----- 1 3.0 0.0 15 201
r 1.35576 0 2 tcp 1000 ----- 1 0.0 3.0 29 199
+ 1.35576 2 3 tcp 1000 ----- 1 0.0 3.0 29 199
d 1.35576 2 3 tcp 1000 ----- 1 0.0 3.0 29 199
+ 1.356 1 2 cbr 1000 ----- 2 1.0 3.1 157 207
- 1.356 1 2 cbr 1000 ----- 2 1.0 3.1 157 207
```

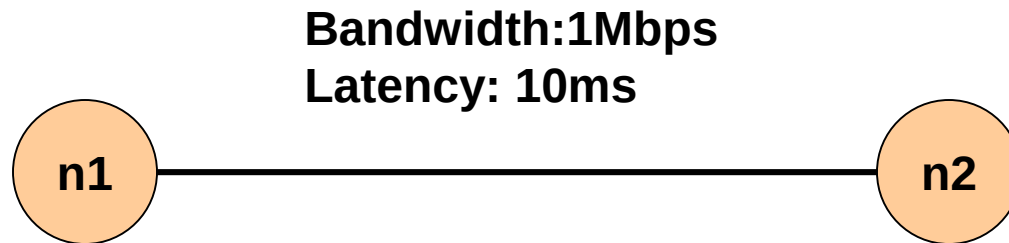
Observing Network Behavior

- NAM:
 - Network Animator
 - A visual aid showing how packets flow along the network
- We'll see a demo..

Basics of using NS2

- Define Network topology, load, output files in Tcl Script
- To run,
 \$ ns simple_network.tcl
- Internally, NS2 instantiates C++ classes based on the tcl scripts
- Output is in form of trace files

A simple Example – Creating the topology



Creating the topology

#create a new simulator object

```
set ns [new Simulator]
```

#open the nam trace file

```
set nf [open out.nam w]
```

```
$ns namtrace-all $nf
```

#define a 'finish' procedure

```
proc finish {} {
```

```
    global ns nf
```

```
    $ns flush-trace
```

#close the trace file

```
close $nf
```

#execute nam on the trace file

```
exec nam out.nam &
```

```
exit 0
```

```
}
```

Creating the topology (Contd)

```
#create two nodes
```

```
set n0 [$ns node]
```

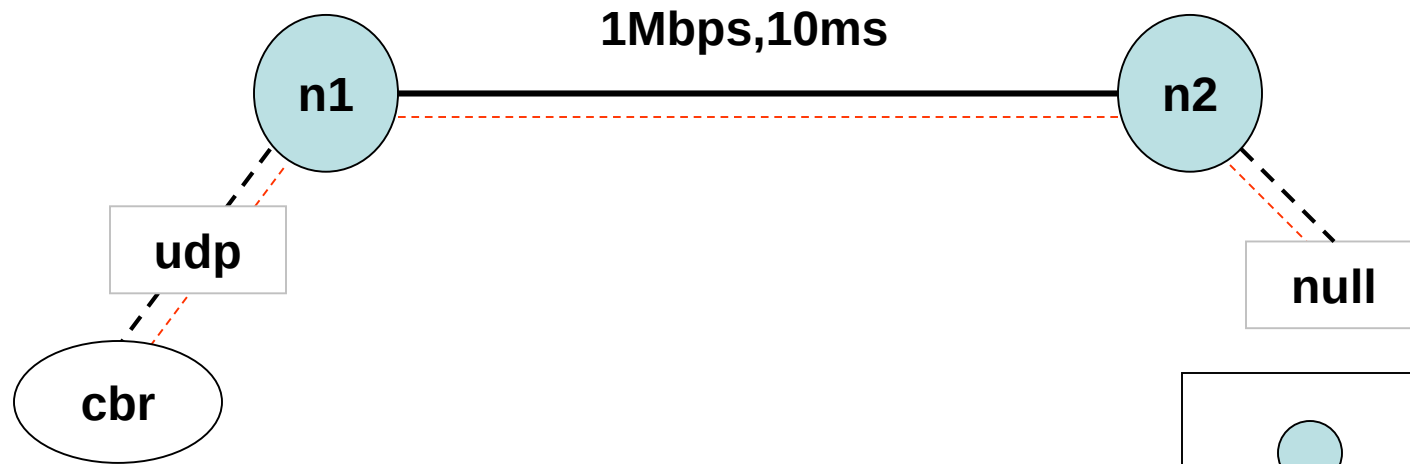
```
set n1 [$ns node]
```

```
#create a duplex link between the nodes
```

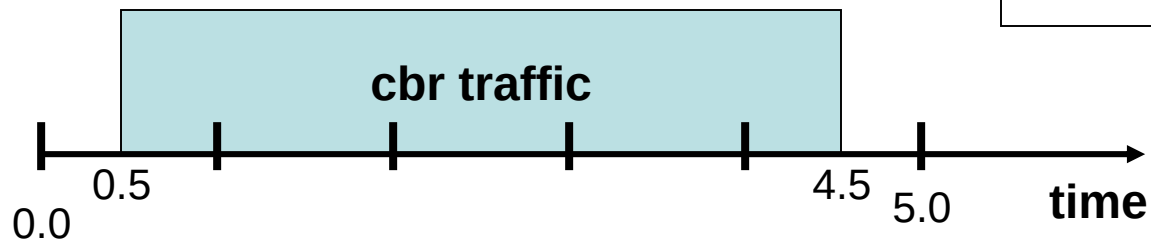
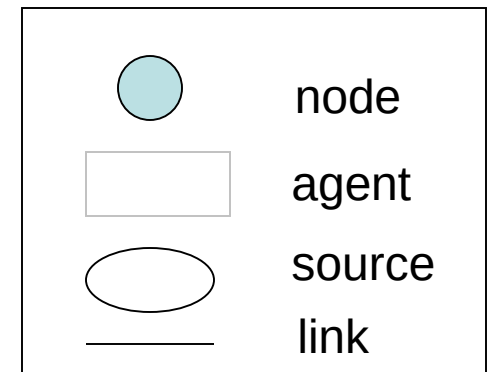
```
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
```

Demo

Adding traffic



Packet Size: 500 bytes
rate: 800Kbps



Putting it together..

#create a udp agent and attach it to node n0

```
set udp0 [new Agent/UDP]
$ns attach-agent $n0 $udp0
```

#Create a CBR traffic source and attach it to udp0

```
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0
```

#create a Null agent(a traffic sink) and attach it to node n1

```
set null0 [new Agent/Null]
$ns attach-agent $n1 $null0
```

#Connect the traffic source to the sink

```
$ns connect $udp0 $null0
```

#Schedule events for CBR traffic

```
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"
```

#call the finish procedure after 5 secs of simulated time

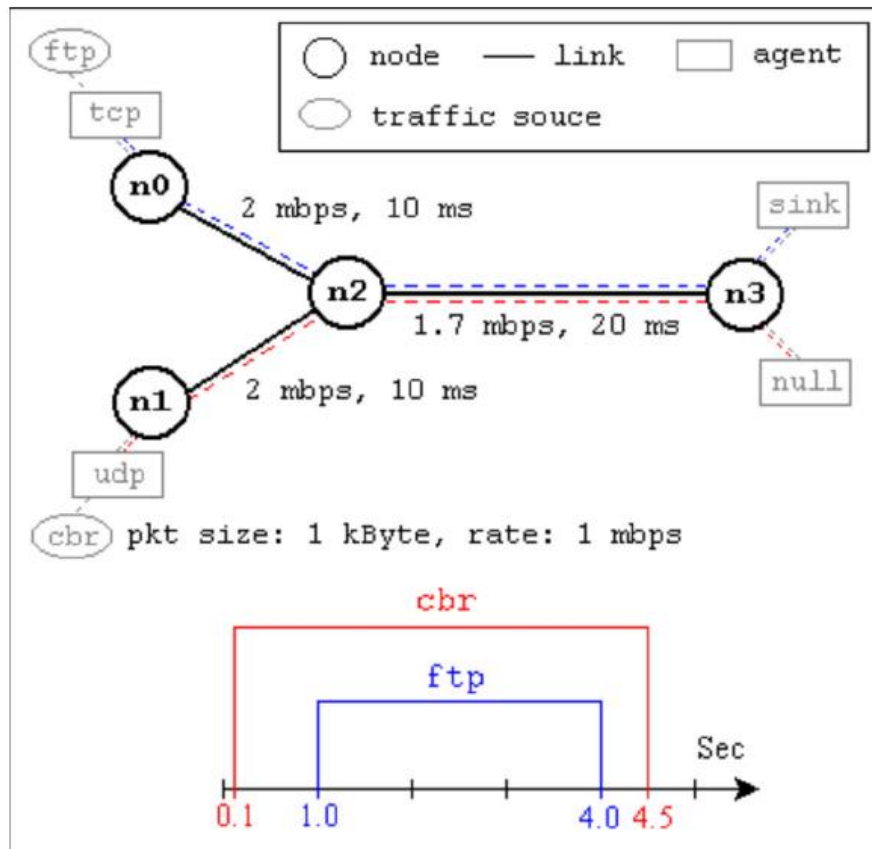
```
$ns at 5.0 "finish"
```

#run the simulation

```
$ns run
```

Demo

A second Scenario * (from NS by Example)



Taken from NS by
Example by [Jae Chung](#)
and
[Mark Claypool](#)

A second Example (From NS by Example)

```
#Create a simulator object
set ns [new Simulator]

#Define different colors for data flows (for NAM)
$ns color 1 Blue
$ns color 2 Red

#Open the NAM trace file
set nf [open out.nam w]
$ns namtrace-all $nf

#Define a 'finish' procedure
proc finish {} {
    global ns nf
    $ns flush-trace
    #Close the NAM trace file
    close $nf
    #Execute NAM on the trace file
    exec nam out.nam &
    exit 0
}
```

A Second Scenario (Contd.)

#Create four nodes

```
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
```

#Create links between the nodes

```
$ns duplex-link $n0 $n2 2Mb 10ms DropTail
$ns duplex-link $n1 $n2 2Mb 10ms DropTail
$ns duplex-link $n2 $n3 1.7Mb 20ms DropTail
```

#Set Queue Size of link (n2-n3) to 10

```
$ns queue-limit $n2 $n3 10
```

A Second Scenario (Contd.)

#Give node position (for NAM)

```
$ns duplex-link-op $n0 $n2 orient right-down  
$ns duplex-link-op $n1 $n2 orient right-up  
$ns duplex-link-op $n2 $n3 orient right
```

#Monitor the queue for link (n2-n3). (for NAM)

```
$ns duplex-link-op $n2 $n3 queuePos 0.5
```

A Second Scenario (Contd.)

#Setup a TCP connection

```
set tcp [new Agent/TCP]
$tcp set class_ 2
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSi
$ns attach-agent $n3 $sin
$ns connect $tcp $sink
$tcp set fid_ 1
```

To create agents or traffic sources, we need to know the class names these objects (Agent/TCP, Agent/TCPSink, Application/FTP and so on).

This information can be found in the NS documentation.

But one shortcut is to look at the "ns-2/tcl/libs/ns-default.tcl" file.

#Setup a FTP over TCP connection

```
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP
```


A Second Scenario (Contd.)

#Setup a UDP connection

```
set udp [new Agent/UDP]
$ns attach-agent $n1 $udp
set null [new Agent/Null]
$ns attach-agent $n3 $null
$ns connect $udp $null
$udp set fid_ 2
```

#Setup a CBR over UDP connection

```
set cbr [new Application/Traffic/CBR]
$cbr attach-agent $udp
$cbr set type_ CBR
$cbr set packet_size_ 1000
$cbr set rate_ 1mb
$cbr set random_ false
```

A Second Scenario (Contd.)

#Schedule events for the CBR and FTP agents

```
$ns at 0.1 "$cbr start"  
$ns at 1.0 "$ftp start"  
$ns at 4.0 "$ftp stop"  
$ns at 4.5 "$cbr stop"
```

#Detach tcp and sink agents (not really necessary)

```
$ns at 4.5 "$ns detach-agent $n0 $tcp ; $ns detach-agent $n3 $sink"
```

#Call the finish procedure after 5 seconds of simulation time

```
$ns at 5.0 "finish"
```

#Print CBR packet size and interval

```
puts "CBR packet size = [$cbr set packet_size_]"  
puts "CBR interval = [$cbr set interval_]"
```

#Run the simulation

```
$ns run
```

Explanation of Code Execution(I)

```
set ns [new Simulator]

set n0 [$ns node]
set n1 [$ns node]

$ns duplex-link $n0 $n1 1Mb 10ms DropTail

set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set sink0 [new Agent/TCPSink]
$ns attach-agent $n1 $sink0
$ns connect $tcp0 $sink0

set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

$ns at 0.5 "$ftp0 start"
$ns at 4.5 "$ftp0 stop"
$ns at 5.0 "stop"

$ns run
```

Simple.tcl

```
Simulator instproc init args {
    $self create_packetformat

    $self use-scheduler Calendar

    $self set nullAgent_ [new Agent/Null]
    $self set-address-format def

    eval $self next $args
}
```

Run your program → %
ns Simple.tcl

Explanation of Code Execution(II)

```
set ns [new Simulator]

set n0 [$ns node]
set n1 [$ns node]

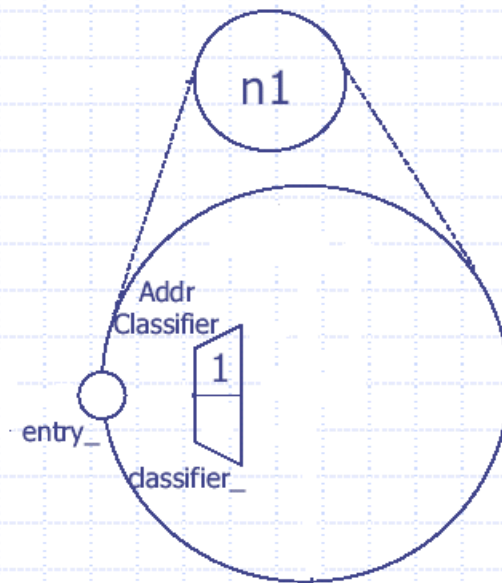
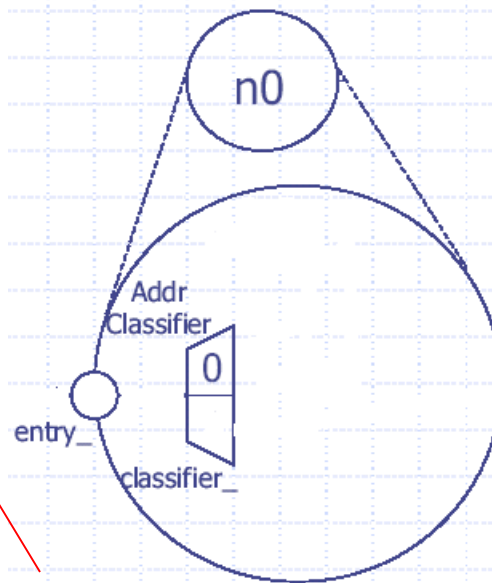
$ns duplex-link $n0 $n1 1Mb 10ms

set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set sink0 [new Agent/TCPSink]
$ns attach-agent $n1 $sink0
$ns connect $tcp0 $sink0

set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

$ns at 0.5 "$ftp0 start"
$ns at 4.5 "$ftp0 stop"
$ns at 5.0 "stop"

$ns run
```



Explanation of Code Execution(III)

```
set ns [new Simulator]

set n0 [$ns node]
set n1 [$ns node]

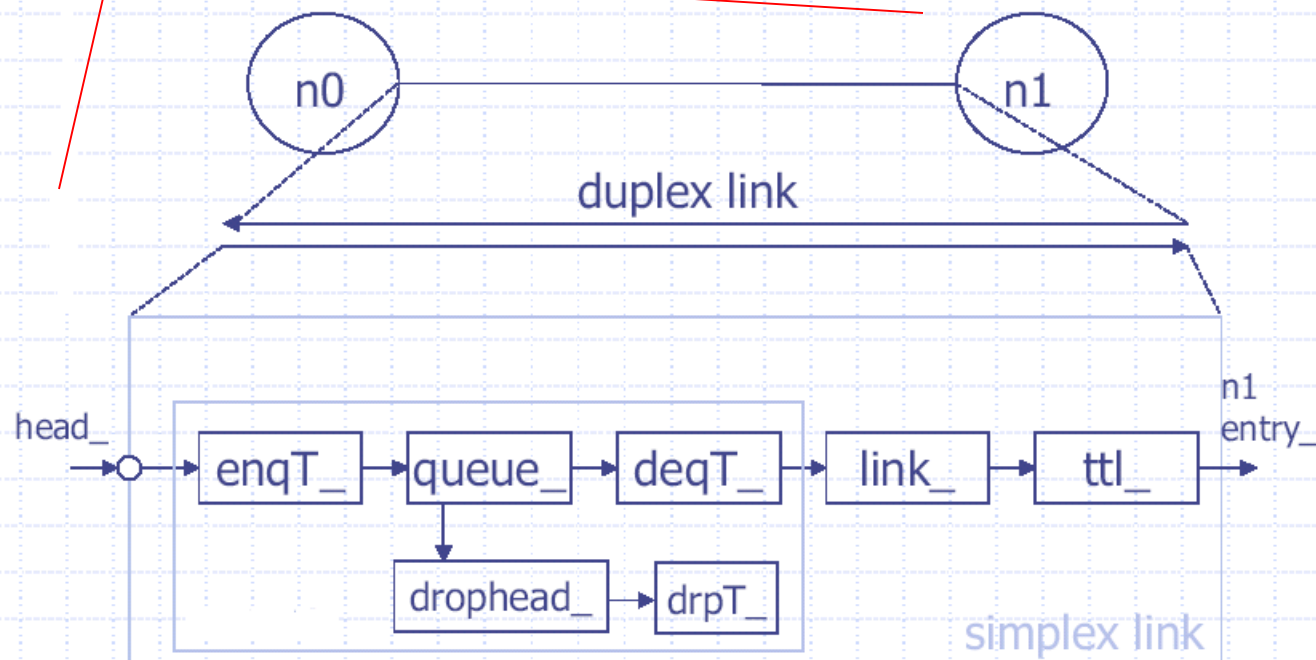
$ns duplex-link $n0 $n1 1Mb 10ms DropTail

set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set sink0 [new Agent/TCP]
$ns attach-agent $n1 $sink0
$ns connect $tcp0 $sink0

set ftp0 [new Application]
$ftp0 attach-agent $tcp0

$ns at 0.5 "$ftp0 start"
$ns at 4.5 "$ftp0 stop"
$ns at 5.0 "stop"

$ns run
```

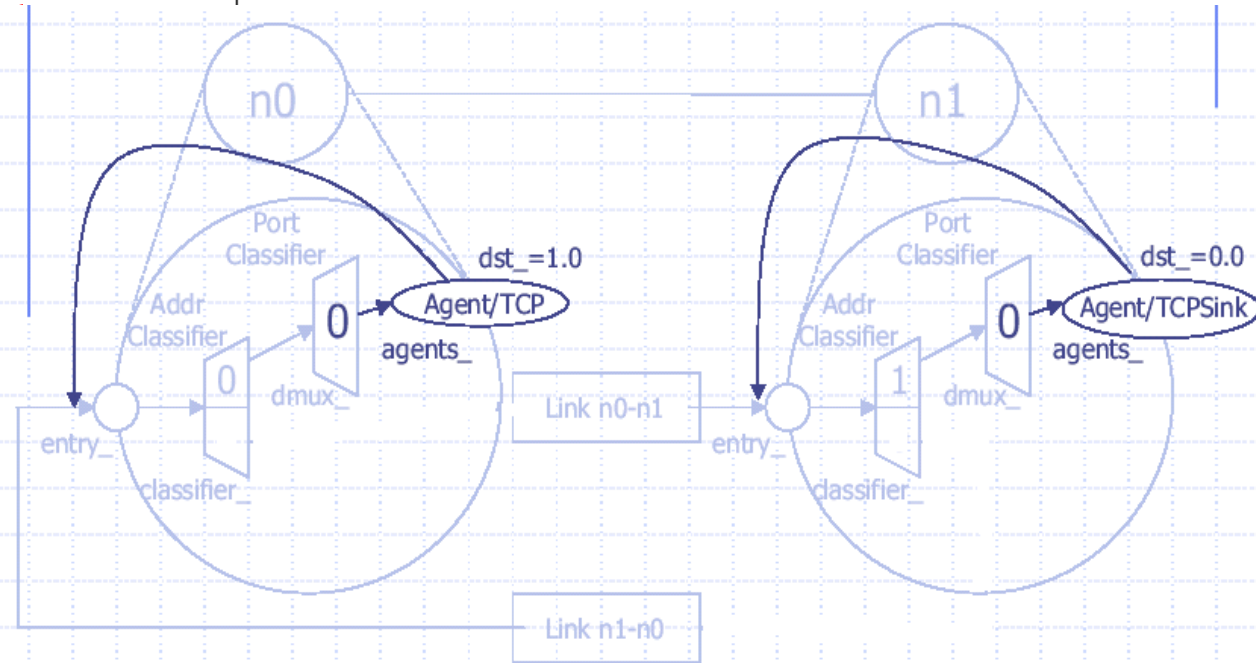


```
set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set sink0 [new Agent/TCPSink]
$ns attach-agent $n1 $sink0
$ns connect $tcp0 $sink0

set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

$ns at 0.5 "$ftp0 start"
$ns at 4.5 "$ftp0 stop"
$ns at 5.0 "stop"

$ns run
```



Explanation of Code Execution(V)

```
set ns [new Simulator]

set n0 [$ns node]
set n1 [$ns node]

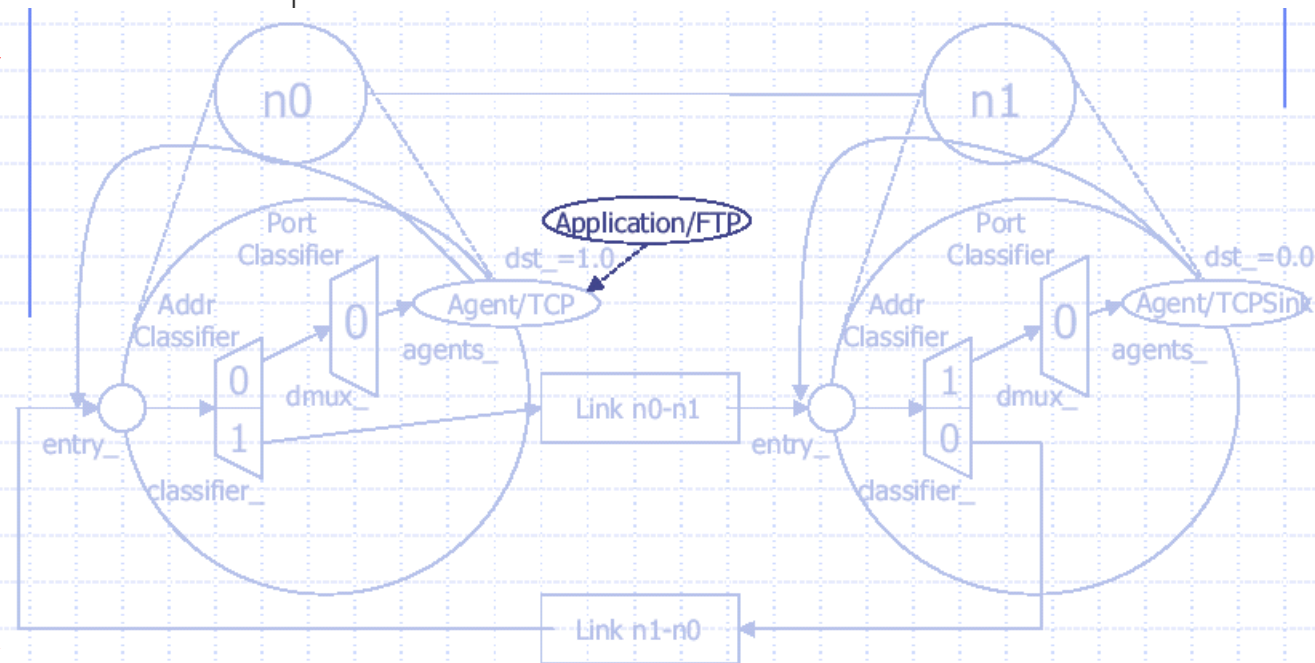
$ns duplex-link $n0 $n1 1Mb 10ms DropTail

set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set sink0 [new Agent/TCPSink]
$ns attach-agent $n1 $sink0
$ns connect $tcp0 $sink0

set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

$ns at 0.5 "$ftp0 start"
$ns at 4.5 "$ftp0 stop"
$ns at 5.0 "stop"

$ns run
```



A thick, dark green horizontal bar with rounded ends, positioned across the upper middle of the slide.

Thank You